

IJ Systems Automation

Automatización del despliegue y administración de servicios en contenedores

Bash · Ansible · Docker · AWS EC2

Ian Gutiérrez Guertsen · Jaime Ferrari Zamorano
I.E.S. Rosa Chacel · Curso 2025/2026



IJ SYSTEMS

Una solución reproducible, validada
y lista para defender



Ruta de la defensa

Una historia clara: problema, diseño, despliegue, pruebas y valor final.



Cada parte de la defensa conecta con una evidencia concreta del proyecto.



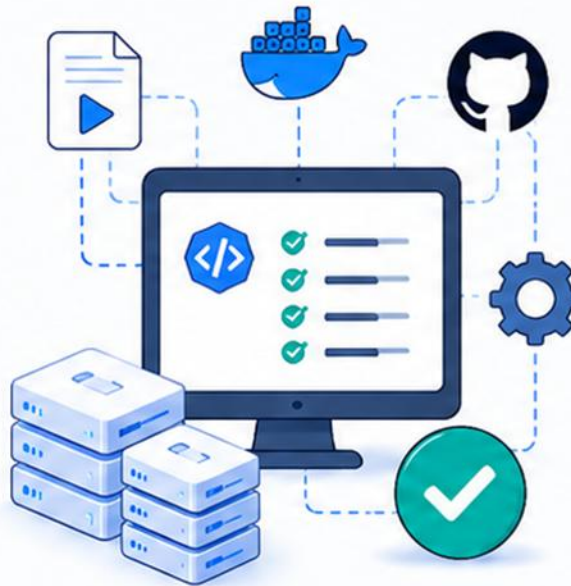
Punto de partida: despliegues manuales

El problema no era instalar una vez, sino repetirlo sin errores.



ANTES

- Instalaciones comando a comando
- Configuraciones inconsistentes
- Dependencia de quien ejecuta
- Recuperación lenta ante incidencias



DESPUÉS

- Servidor preparado con scripts y playbooks
- Servicios definidos en Docker Compose
- Configuración versionada en GitHub
- Pruebas y recuperación documentadas



Reto técnico

Convertir tareas manuales en un procedimiento reproducible, auditable y transferible.

Solución elegida

Arquitectura modular: cada capa cubre una responsabilidad concreta



INFRAESTRUCTURA

Servidor base en la nube



Amazon EC2



Ubuntu Server LTS



prepara



AUTOMATIZACIÓN

Configuración y despliegue reproducibles



Bash



Ansible



Docker Compose



orquesta



SERVICIOS

Contenedores publicados y supervisados



Nginx



MariaDB



Prometheus



cAdvisor



Uptime Kuma

Decisión clave: no sobredimensionar; automatizar bien lo necesario



Objetivos medibles y trazabilidad

Cada objetivo SMART queda validado con pruebas concretas



OBJ-01

Preparar Ubuntu
en una ejecución

→ CP-07



OBJ-02

Desplegar pila
en < 3 min

→ CP-01 · CP-02
CP-04 · CP-05 · CP-06



OBJ-03

Validar Nginx
desde Internet

→ CP-03 · CP-14



OBJ-04

Backup y restore
MariaDB

→ CP-08 · CP-09
CP-10 · CP-11 · CP-13



OBJ-05

Trazabilidad
en GitHub

→ CP-12

Trazabilidad completa: requisito → implementación → prueba → evidencia



Servicios desplegados

Una pila ligera para demostrar publicación, persistencia, métricas y supervisión.



Nginx

Servidor web · Publicación HTTP

PUBLICACIÓN



MariaDB

Base de datos · Volumen persistente

PERSISTENCIA



Prometheus

Recogida de métricas técnicas

MONITORIZACIÓN



cAdvisor

Métricas de contenedores Docker

MONITORIZACIÓN



node-exporter

Métricas del host hacia Prometheus

MONITORIZACIÓN



Uptime Kuma

Supervisión de disponibilidad

SUPERVISIÓN

Cada contenedor tiene una función clara dentro del despliegue





Implementación en AWS EC2

Infraestructura ligera, acceso SSH seguro y reglas de entrada controladas.



Instancia EC2

- ✓ Ubuntu Server LTS (22.04)
- ✓ Instancia t3.small (2 vCPU, 2 GB RAM)
- ✓ Almacenamiento EBS 20 GB (gp3)
- ✓ Entorno base listo para desplegar



Entorno preparado y optimizado para la ejecución de servicios.



Acceso SSH

- ✓ Acceso seguro mediante clave SSH
- ✓ Usuario ubuntu con permisos mínimos
- ✓ Deshabilitado acceso por contraseña
- ✓ Conexión cifrada y protegida



Acceso seguro y controlado bajo buenas prácticas.



Security Group

- ✓ Solo tráfico necesario permitido
- ✓ Puerto 22 (SSH) desde IP autorizada
- ✓ Puertos específicos ajustados según servicio
- ✓ Todo lo demás denegado (principio de mínimo privilegio)



Superficie de ataque mínima y entorno más seguro.



Criterio defendible

La nube se usó de forma puntual y controlada: suficiente para validar el proyecto sin convertirlo en un servicio 24/7.



Bajo coste operativo



Alta disponibilidad de AWS



Despliegue rápido y reproducible



Monitorizable y escalable

Seguridad básica y gestión de credenciales

Menor exposición, credenciales fuera del repositorio y reglas controladas.



Exposición controlada

Solo se exponen los servicios estrictamente necesarios.



SSH 22

solo IP del administrador



Nginx 80

servicio público de validación



9090 / 3001 / 8080

solo IP del administrador



MariaDB 3306

no expuesto públicamente



Gestión de credenciales

Las credenciales no se almacenan en el repositorio.

.env local



- ✓ docker/.env se mantiene local
- ✓ Se excluye del control de versiones
- ✓ Contiene credenciales reales y sensibles



.env.example



- ✓ Se sustituye por .env.example
- ✓ Sin contraseñas reales ni datos sensibles



Las credenciales quedan protegidas y el proyecto sigue siendo reproducible.



Principio aplicado

mínima exposición + trazabilidad + credenciales protegidas

Seguridad básica suficiente para el alcance académico del proyecto.



Troubleshooting

Incidencias reales del despliegue: causa, corrección y resultado verificado



Método aplicado: detectar síntoma • localizar causa • aplicar corrección • verificar resultado



5 incidencias documentadas: causa probable, corrección aplicada y estado final correcto

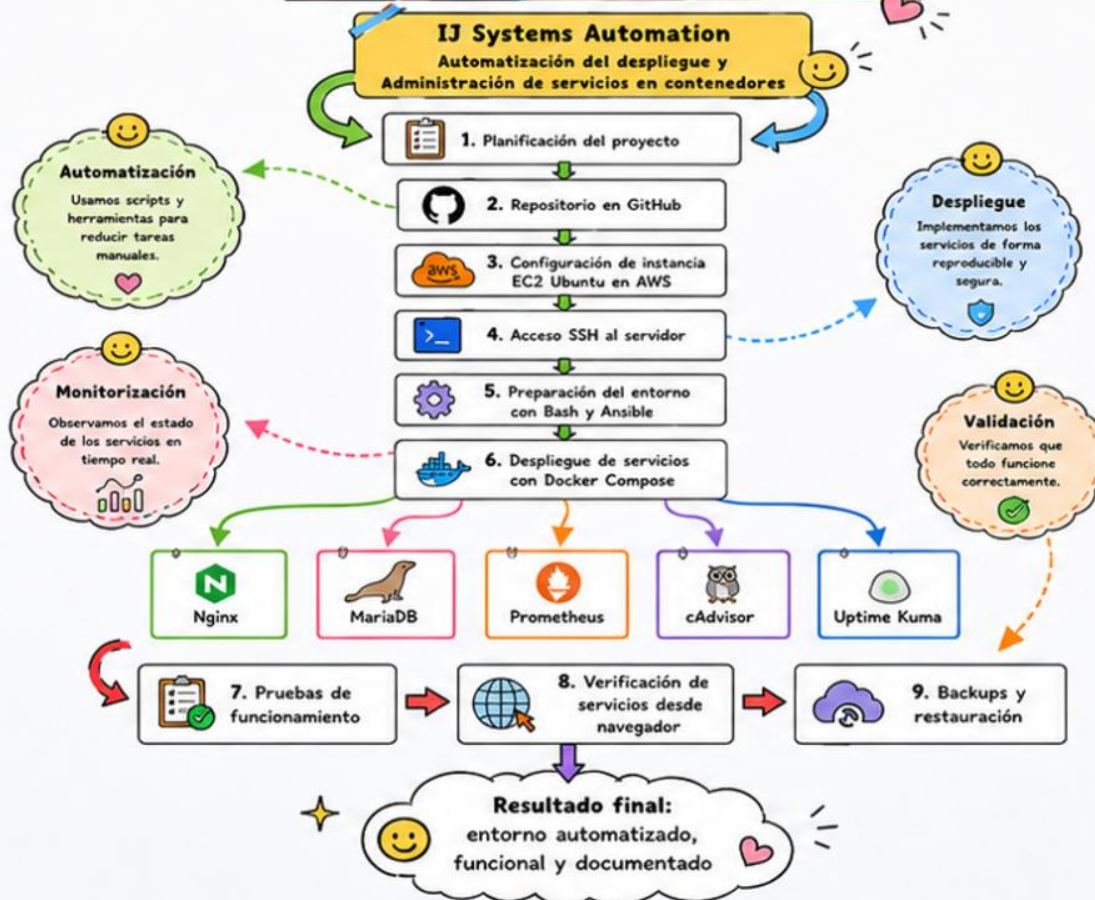


Flujo de trabajo completo

De la planificación a la validación y recuperación del entorno.



Diagrama de flujo del proyecto



Secuencia defendible

- 1 Planificar y versionar el proyecto
- 2 Crear EC2 y acceder por SSH
- 3 Preparar el host con Bash y Ansible
- 4 Levantar la pila con Docker Compose
- 5 Validar web, métricas y persistencia
- 6 Cerrar con backup, restore y documentación

Validación: pruebas con evidencias

Cada caso de prueba con su evidencia visual y resultado verificado



CP-03

✓

Nginx publicado en AWS

CP-04

✓

Targets activos en Prometheus

CP-08

Persistencia MariaDB

- ✓ Tablas y datos presentes
- ✓ Registros activos
- ✓ Integridad referencial correcta

✓

Persistencia MariaDB validada

CP-09 / CP-10

Backup y restore

- ✓ Backup generado (.sql)
- ✓ Tamaño: 12M
- ✓ Restore completado
- ✓ Servicio operativo tras restore

✓

Backup y restore funcionales

14/14

casos de prueba superados

✓

Infraestructura y despliegue

CP-01 · CP-02 · CP-07

3/3

Servicios publicados y proxy

CP-03 · CP-04 · CP-05 · CP-06 · CP-14

5/5

Persistencia y recuperación

CP-08 · CP-09 · CP-10 · CP-11 · CP-13

5/5

Trazabilidad

CP-12

1/1

100% de los objetivos validados con evidencia

Recuperación: backup y restore de MariaDB

Una prueba especialmente defendible: escribir, perder y recuperar datos.



Validación de datos

- ✓ Inserción y consulta en MariaDB
- ✓ Dato visible antes del backup
- ✓ Prueba funcional correcta



Copia creada

- ✓ Ejecución de backup.sh
- ✓ Archivo .sql generado
- ✓ Copia guardada correctamente



Restauración correcta

- ✓ Ejecución de restore.sh
- ✓ Dato recuperado
- ✓ Servicio operativo tras restore



Objetivo

Demostrar que la base de datos puede recuperarse sin rehacer el despliegue.



Proceso

Backup, borrado controlado y restauración documentados con evidencias.



Resultado

Persistencia validada y recuperación correcta de MariaDB.

Resultados obtenidos

Un entorno automatizado, documentado y validado de extremo a extremo



Tiempo operativo: manual vs automatizado



-80%

tiempo operativo

frente al procedimiento manual

< 3 min

Despliegue de la pila

docker compose up -d

14/14

Casos de prueba

con evidencias documentadas

5/5

Objetivos SMART

OBJ-01 a OBJ-05 cubiertos

0

Requisitos pendientes

alcance completado



Cualquier técnico puede reproducir el despliegue desde cero con el repositorio



Viabilidad económica

Coste real durante el desarrollo y estimación fuera de AWS Academy



COSTE REAL DEL PROYECTO

0,77 USD

≈ **menos de 1 dólar**

para 50 horas de pruebas reales en AWS

Cómputo EC2 t3.small + IP elástica · cálculo orientativo



DURANTE EL DESARROLLO

≈ **50 €**

crédito AWS Academy por cuenta ·
2 cuentas disponibles



ALTERNATIVA 24/7

12,69 USD/mes

uso continuo fuera de AWS Academy



GASTO ADICIONAL REAL

0 €

todo cubierto por créditos académicos



Viable para laboratorio, formación o pequeña empresa sin inversión significativa



Ampliaciones futuras

Mejoras naturales si el proyecto evoluciona a un entorno más profesional.



HTTPS/TLS

Certificados y
publicación segura



IaC

Terraform o
CloudFormation



Alertas

Notificaciones de caída
o degradación



Backups programados

Rotación, retención y
pruebas periódicas



Escalado

Alta disponibilidad o Kubernetes
si el tamaño lo justifica



HashiCorp Vault

Gestión centralizada
de secretos



AWS Secrets Manager

Servicio gestionado
de AWS



Objetivo

Seguir reforzando la seguridad, la
resiliencia y la escalabilidad del sistema.



Beneficio

Infraestructura más robusta, segura y
preparada para crecer.



Impacto

Mayor fiabilidad, mejor rendimiento y
operaciones más eficientes.

La automatización no es escribir scripts.

Es convertir tareas manuales en procedimientos reproducibles, auditables y transferibles.

Preparados para preguntas sobre:

Seguridad

Docker Compose

Backup / Restore

Costes

Ampliaciones

Ian Gutiérrez Guertsen · Jaime Ferrari Zamorano

I.E.S. Rosa Chacel · ASIR · Curso 2025/2026

GRACIAS

EXPLORA EL PROYECTO



Repositorio GitHub



github.com/Quontyx/IJ-Systems-Automation

